# Lumen API – dokumentacja

Endpointy publiczne: `/health`, `/products`, `/cart/quote`, `/checkout`, `/webhooks/stripe`, `/auth/*`
Endpointy klienta (sesja): `/account/*`
Endpointy panelu administracyjnego (sesja z rolą `ADMIN`): `/admin/*`

```
https://lumen.com/api          (przez aplikację web, rewrite Next.js)
http://localhost:4300          (lokalnie, bezpośrednio do API)
```

### Spis treści


1. Autoryzacja
2. Konwencje i błędy
3. Wspólne obiekty: produkt i zamówienie
4. GET /health
5. GET /products
6. GET /products/:slug
7. POST /cart/quote
8. POST /checkout
9. GET /checkout/confirmation
0. POST /webhooks/stripe
11. POST /auth/register
12. POST /auth/login · logout · GET /auth/me
13. POST /auth/password-reset/*
14. GET /account/orders · /account/orders/:id
15. /account/addresses
16. GET /admin/stats
17. GET /admin/products · PATCH products · variants
18. GET /admin/orders · /admin/orders/:id
19. PATCH /admin/orders/:id/status


## Autoryzacja

Katalog, wycena koszyka, checkout i webhook nie wymagają logowania. Zakupy działają też jako gość. Endpointy `/account/*` wymagają sesji klienta, a `/admin/*` sesji użytkownika z rolą `ADMIN`. Uprawnienia sprawdza wyłącznie API, przy każdym żądaniu i dla każdego rekordu (klient widzi tylko swoje zamówienia i adresy).

Sesję otwiera `POST /auth/login` albo `POST /auth/register`. Oba ustawiają ciasteczko:

| Ciasteczko | Opis |
|---|---|
| `lumen_session` | Losowy token (32 bajty). W bazie zapisany jest tylko jego skrót SHA-256. Flagi: `HttpOnly`, `SameSite=Lax`, `Path=/`, `Secure` na produkcji. Ważne 30 dni. Reset hasła unieważnia wszystkie sesje. |

Wymagania dla żądań:

| Nagłówek | Opis |
| --- | --- |
| `Cookie` | `lumen_session=…`. Przeglądarka dołącza je sama (to samo pochodzenie dzięki rewrite `/api/*`), w curl: `-b`. |
| `Content-Type` | `application/json` dla każdego żądania z treścią (POST, PUT, PATCH). |
| `Origin` | Opcjonalny. Jeśli jest wysłany przy POST/PUT/PATCH/DELETE, musi wskazywać `WEB_URL` albo adres z `ALLOWED_ORIGINS`, inaczej `403 origin_not_allowed`. |

Przykład (curl): logowanie i pierwsze żądanie z tą samą sesją:

```
curl -c cookies.txt -H 'Content-Type: application/json' \
  -d '{"email":"anna.nowak@example.com","password":"…"}' \
  https://lumen.com/api/auth/login

curl -b cookies.txt https://lumen.com/api/account/orders
```

Brak sesji → `401`, `{"error": "unauthorized"}`. Sesja bez roli admina na `/admin/*` → `403`, `{"error": "forbidden"}`.

| Limit | Wartość |
| --- | --- |
| Globalny | 300 żądań na minutę z jednego IP |
| `/auth/register`, `/auth/login`, `/auth/password-reset/*` | 5 żądań na minutę |
| `/checkout` | 10 żądań na minutę |
| `/checkout/confirmation` | 30 żądań na minutę |
| `POST /account/addresses` | 20 żądań na minutę |
| `/cart/quote` | 60 żądań na minutę |
| `/webhooks/stripe` | bez limitu (podpis Stripe) |

Przekroczenie limitu:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60

{
  "error": "rate_limited"
}
```

## Konwencje i błędy

| Zasada | Opis |
| --- | --- |
| Format | JSON w UTF-8. Wyjątek: `DELETE /account/addresses/:id` zwraca `204` bez treści. |
| Ceny | Liczby całkowite w groszach (PLN), pola z końcówką `Grosze`: `15900` = 159,00 zł. Cenę zawsze liczy serwer z bazy, klient wysyła tylko `variantId` i ilość. |
| Daty | ISO 8601 w UTC, np. `2026-10-11T14:22:58.820Z`. |
| Identyfikatory | CUID (`cmv54jvex00005e8obintkd10`). Produkty mają też `slug` (`cedr-i-dym`), warianty `sku` (`LUM-CEDR-I-DYM-180`), zamówienia numer `LM-RRMMDD-XXXX`. |
| Język | Parametr `locale`: `pl` (domyślny) albo `en`. Wpływa na nazwy, opisy i alt zdjęć. |
| Dostawa | `COURIER` 19,99 zł, `PARCEL_LOCKER` 14,99 zł, darmowa od 300 zł. Tylko Polska. |
| Honeypot | Formularze publiczne mają pole `website`. Musi być puste albo pominięte. |
| Maks. rozmiar treści | 64 kB na żądanie. |

Każdy błąd ma pole `error` z kodem. Błąd walidacji ma dodatkowo listę `issues` (ścieżka pola, kod, opis):

```
{
  "error": "validation_error",
  "issues": [
    {
      "path": "postalCode",
      "code": "invalid_format",
      "message": "Invalid string: must match pattern /^\\d{2}-\\d{3}$/"
    },
    {
      "path": "phone",
      "code": "invalid_format",
      "message": "Invalid string: must match pattern /^\\+?[0-9 ()-]{9,20}$/"
    }
  ]
}
```

| Kod | Kiedy |
| --- | --- |
| `400` | `validation_error` (z `issues`) albo kod biznesowy, np. `invalid_token`, `invalid_transition`, `tracking_required`. |
| `401` | `unauthorized` (brak sesji) albo `invalid_credentials` (złe dane logowania). |
| `403` | `forbidden` (brak roli admina) albo `origin_not_allowed` (obcy `Origin`). |
| `404` | Nie ma zasobu (`product_not_found`, `order_not_found`, `address_not_found`, `variant_not_found`) albo endpointu (`not_found`). |
| `409` | Konflikt: `email_taken`, `cart_changed` (brak towaru przy checkoutcie), `address_limit` (maks. 10 adresów). |
| `429` | `rate_limited`, nagłówek `Retry-After` w sekundach. |
| `500` | `internal_error`. Szczegóły tylko w logu serwera. |
| `503` | `payments_unavailable`: brak kluczy Stripe na serwerze. |

## Wspólne obiekty: produkt i zamówienie

### Obiekt produktu (lista)

Zwracany przez `GET /products`. `GET /products/:slug` zwraca ten sam obiekt rozszerzony o pola opisane przy tym endpoincie.

### Znaczenie kluczy w odpowiedzi

<table>
<tr><th>Klucz</th><th>Opis</th></tr>
<tr><td><code>id</code>, <code>slug</code></td><td>Identyfikator i adres produktu.</td></tr>
<tr><td><code>name</code>, <code>tagline</code></td><td>Nazwa i hasło w wybranym języku.</td></tr>
<tr><td><code>family</code></td><td>Rodzina zapachu:<table><tr><th>Wartość</th><th>Znaczenie</th></tr><tr><td><code>WOODY</code></td><td>drzewne</td></tr><tr><td><code>FLORAL</code></td><td>kwiatowe</td></tr><tr><td><code>CITRUS</code></td><td>cytrusowe</td></tr><tr><td><code>GOURMAND</code></td><td>gurmańskie</td></tr><tr><td><code>FRESH</code></td><td>świeże</td></tr><tr><td><code>SPICY</code></td><td>korzenne</td></tr></table></td></tr>
<tr><td><code>accentColor</code></td><td>Kolor szkła w hex, używany jako akcent w UI.</td></tr>
<tr><td><code>fromPriceGrosze</code></td><td>Najniższa cena spośród aktywnych wariantów.</td></tr>
<tr><td><code>notes.top / heart / base</code></td><td>Nuty głowy, serca i bazy.</td></tr>
<tr><td><code>images[]</code></td><td>Zdjęcia z rolą: <code>COLOR</code> (tło w kolorze zapachu), <code>NEUTRAL</code> (jasne tło, świeca zgaszona), <code>TOP</code> (z góry). Każde ma <code>url</code>, <code>alt</code>, <code>width</code>, <code>height</code>.</td></tr>
<tr><td><code>available</code></td><td><code>true</code>, gdy którykolwiek wariant jest na stanie.</td></tr>
<tr><td><code>featured</code></td><td>Wyróżniony na stronie głównej.</td></tr>
<tr><td><code>defaultVariant</code></td><td>Najmniejszy dostępny wariant (używany przez szybkie dodawanie): <code>id</code>, <code>sku</code>, <code>label</code>, <code>weightGrams</code>, <code>burnHours</code>, <code>priceGrosze</code>, <code>stockStatus</code>.</td></tr>
</table>

| | |
|---|---|
| `stockStatus` | <table><tr><th>Wartość</th><th>Znaczenie</th></tr><tr><td>`IN_STOCK`</td><td>więcej niż 5 sztuk</td></tr><tr><td>`LOW_STOCK`</td><td>1-5 sztuk</td></tr><tr><td>`OUT_OF_STOCK`</td><td>brak</td></tr></table> |
| `variantCount` | Liczba aktywnych wariantów. |

### Obiekt zamówienia

Zwracany przez `/account/orders`, `/checkout/confirmation` i `/admin/orders/:id`.

```
{
  "id": "cmv70dzwm000e3e8oco6p858w",
  "number": "LM-261011-G5QE",
  "status": "PAID",
  "email": "anna.nowak@example.com",
  "createdAt": "2026-10-11T14:22:58.820Z",
  "paidAt": "2026-10-11T14:23:58.820Z",
  "subtotalGrosze": 72700,
  "shippingGrosze": 0,
  "totalGrosze": 72700,
  "trackingNumber": null,
  "shipping": {
    "method": "COURIER",
    "parcelLockerCode": null,
    "firstName": "Anna",
    "lastName": "Nowak",
    "line1": "ul. Przykładowa 12/4",
    "line2": "",
    "postalCode": "00-950",
    "city": "Warszawa",
    "country": "PL",
    "phone": "+48 600 100 200"
  },
  "items": [
    {
      "productName": "Tytoń i Wanilia",
      "variantLabel": "180 g",
      "unitPriceGrosze": 16900,
      "quantity": 1
    },
    {
      "productName": "Ambra Nocturna",
      "variantLabel": "380 g",
      "unitPriceGrosze": 27900,
      "quantity": 2
    }
  ]
}
```

### Znaczenie kluczy w odpowiedzi

<table>
<tr><th>Klucz</th><th>Opis</th></tr>
<tr><td><code>number</code></td><td>Numer dla klienta, np. <code>LM-261011-G5QE</code>.</td></tr>
<tr><td><code>status</code></td><td>Status zamówienia:
<table>
<tr><th>Wartość</th><th>Znaczenie</th></tr>
<tr><td><code>PENDING</code></td><td>utworzone, czeka na płatność (nie widać go na koncie klienta ani na domyślnej liście w panelu)</td></tr>
<tr><td><code>PAID</code></td><td>opłacone, do wysłania</td></tr>
<tr><td><code>SHIPPED</code></td><td>wysłane, ma numer przesyłki</td></tr>
<tr><td><code>DELIVERED</code></td><td>doręczone</td></tr>
<tr><td><code>CANCELLED</code></td><td>anulowane: w panelu, automatycznie po 24 h bez płatności albo po anulowaniu PaymentIntent w Stripe</td></tr>
<tr><td><code>FAILED</code></td><td>płatność odrzucona</td></tr>
</table></td></tr>
<tr><td><code>paidAt</code></td><td>Moment zaksięgowania płatności albo <code>null</code>.</td></tr>
<tr><td><code>subtotalGrosze / shippingGrosze / totalGrosze</code></td><td>Wartość produktów, dostawy i razem.</td></tr>
<tr><td><code>trackingNumber</code></td><td>Numer przesyłki InPost po wysyłce.</td></tr>
<tr><td><code>shipping</code></td><td>Metoda (<code>COURIER</code> / <code>PARCEL_LOCKER</code>), kod paczkomatu i adres.</td></tr>
<tr><td><code>items[]</code></td><td>Pozycje z nazwą i ceną z chwili zakupu (późniejsze zmiany cen ich nie zmieniają).</td></tr>
</table>

# GET /health

**GET** `/health`

Sprawdza, czy API działa i ma połączenie z bazą. Do monitoringu i health checków hostingu.

### Przykład żądania

```
GET http://localhost:4300/health
```

### Odpowiedź – sukces

```
{
  "status": "ok"
}
```

Gdy baza nie odpowiada, endpoint zwraca `500` z `{"error": "internal_error"}`.

# GET /products

`GET` `/products`

Lista opublikowanych produktów (status `ACTIVE`) w kolejności z panelu.

### Query params

| Parametr | Wymagany | Opis |
|---|---|---|
| `locale` | nie | `pl` (domyślnie) albo `en`. |
| `family` | nie | Filtr rodziny zapachu:<br>`WOODY` — drzewne<br>`FLORAL` — kwiatowe<br>`CITRUS` — cytrusowe<br>`GOURMAND` — gurmańskie<br>`FRESH` — świeże<br>`SPICY` — korzenne |

### Przykład żądania

```
GET https://lumen.com/api/products?locale=pl&family=WOODY
```

### Odpowiedź – błąd walidacji (400)

```
{
  "error": "validation_error",
  "issues": [
    {
      "path": "family",
      "code": "invalid_value",
      "message": "Invalid option: expected one of
\"WOODY\"|\"FLORAL\"|\"CITRUS\"|\"GOURMAND\"|\"FRESH\"|\"SPICY\""
    }
  ]
}
```

### Odpowiedź – sukces (skrócony przykład)


```
[
  {
    "id": "cmv54jvex00005e8obintkd10",
    "slug": "ambra-nocturna",
    "name": "Ambra Nocturna",
    "tagline": "Ciepło bursztynu po zmroku",
    "family": "WOODY",
    "accentColor": "#8A5A3C",
    "fromPriceGrosze": 16900,
    "notes": {
      "top": [
        "różowy pieprz",
        "bergamota"
      ],
      "heart": [
        "labdanum",
        "ambra"
      ],
      "base": [
        "wanilia burbońska",
        "drzewo sandałowe"
      ]
    },
    "images": [
      {
        "role": "COLOR",
        "url": "/products/ambra-nocturna-color.webp",
        "alt": "Świeca Ambra Nocturna w barwionym szkle na kolorowym tle",
        "width": 1200,
        "height": 1500
      },
      {
        "role": "NEUTRAL",
        "url": "/products/ambra-nocturna-neutral.webp",
        "alt": "Świeca Ambra Nocturna na jasnym tle",
        "width": 1200,
        "height": 1500
      },
      {
        "role": "TOP",
        "url": "/products/ambra-nocturna-top.webp",
        "alt": "Świeca Ambra Nocturna z góry, widoczny wosk i płonący knot",
        "width": 1200,
        "height": 1500
      }
    ],
    "available": true,
```

```
    "featured": true,
    "defaultVariant": {
      "id": "cmv54jvfz00035e8orvm7uhqo",
      "sku": "LUM-AMBRA-NOCTURNA-180",
      "label": "180 g",
      "weightGrams": 180,
      "burnHours": 45,
      "priceGrosze": 16900,
      "stockStatus": "IN_STOCK"
    },
    "variantCount": 2
  },
  "…"
]
```


Pola obiektu opisuje sekcja Obiekt produktu.

# GET /products/:slug

GET `/products/:slug`

Pełna karta produktu: opis, nastrój, rytuał i wszystkie aktywne warianty.

### Parametry ścieżki

| Parametr | Wymagany | Opis |
|---|---|---|
| `slug` | tak | Slug produktu: małe litery, cyfry i myślniki, do 80 znaków. |

### Query params

| Parametr | Wymagany | Opis |
|---|---|---|
| `locale` | nie | `pl` albo `en`. |

### Przykład żądania

```
GET https://lumen.com/api/products/cedr-i-dym?locale=pl
```

### Odpowiedź – błąd (404)

```
{
  "error": "product_not_found"
}
```

### Odpowiedź – sukces


```
{
  "id": "cmv54jvg200055e8ouh97xfoq",
  "slug": "cedr-i-dym",
  "name": "Cedr i Dym",
  "tagline": "Kominek w górskiej chacie",
  "family": "WOODY",
  "accentColor": "#5B5247",
  "fromPriceGrosze": 15900,
  "notes": {
    "top": [
      "kardamon",
      "jałowiec"
    ],
    "heart": [
      "cedr atlaski",
      "skóra"
    ],
    "base": [
      "wetiweria",
      "dziegieć brzozowy"
    ]
  },
  "images": [
    {
      "role": "COLOR",
      "url": "/products/cedr-i-dym-color.webp",
      "alt": "Świeca Cedr i Dym w barwionym szkle na kolorowym tle",
      "width": 1200,
      "height": 1500
    },
    "…"
  ],
  "available": true,
  "featured": true,
  "defaultVariant": {
    "id": "cmv54jvg800085e8opzvup5mw",
    "sku": "LUM-CEDR-I-DYM-180",
    "label": "180 g",
    "weightGrams": 180,
    "burnHours": 45,
    "priceGrosze": 15900,
    "stockStatus": "IN_STOCK"
  },
  "variantCount": 2,
  "description": "Suchy cedr atlaski, wetiweria i dziegieć brzozowy. Zapach drewna, które dopiero co zgasło, z nutą skóry i chłodnego powietrza za oknem.",
  "mood": "Gruby sweter, ciężka książka, śnieg za oknem",
```

```
  "ritual": "Najlepsza w chłodne popołudnia, w dużym pokoju z wysokim sufitem",
  "variants": [
    {
      "id": "cmv54jvg800085e8opzvup5mw",
      "sku": "LUM-CEDR-I-DYM-180",
      "label": "180 g",
      "weightGrams": 180,
      "burnHours": 45,
      "priceGrosze": 15900,
      "stockStatus": "IN_STOCK"
    },
    {
      "id": "cmv54jvga00095e8ohu5xco8u",
      "sku": "LUM-CEDR-I-DYM-380",
      "label": "380 g",
      "weightGrams": 380,
      "burnHours": 85,
      "priceGrosze": 26900,
      "stockStatus": "IN_STOCK"
    }
  ]
}
```


### Znaczenie kluczy w odpowiedzi

| Klucz | Opis |
|---|---|
| `description` | Opis zapachu. |
| `mood`, `ritual` | Nastrój i kiedy zapalać. |
| `variants[]` | Wszystkie aktywne rozmiary, pola jak w `defaultVariant`. |
| pozostałe | Jak w obiekcie produktu. |

# POST /cart/quote

**POST** `/cart/quote`

Wycenia koszyk po stronie serwera. Koszyk w przeglądarce trzyma tylko `variantId` i ilość, a ceny, nazwy i dostępność zawsze pochodzą z tej odpowiedzi.

### Treść żądania (JSON)

| Parametr | Wymagany | Opis |
|---|---|---|
| `locale` | tak | `pl` albo `en`. |
| `items[]` | tak | Do 20 pozycji: `variantId` i `quantity` (1-10). |

### Przykład żądania

```
POST https://lumen.com/api/cart/quote
Content-Type: application/json

{
  "locale": "pl",
  "items": [
    {
      "variantId": "cmv54jvg800085e8opzvup5mw",
      "quantity": 2
    },
    {
      "variantId": "cmv54jvfz00035e8orvm7uhqo",
      "quantity": 1
    },
    {
      "variantId": "nieistniejacy",
      "quantity": 1
    }
  ]
}
```

### Odpowiedź – błąd walidacji (400)

```
{
  "error": "validation_error",
  "issues": [
    {
      "path": "items.0.quantity",
      "code": "too_big",
      "message": "Too big: expected number to be <=10"
    }
  ]
}
```

### Odpowiedź – sukces


```
{
  "lines": [
    {
      "variantId": "cmv54jvg800085e8opzvup5mw",
      "productSlug": "cedr-i-dym",
      "productName": "Cedr i Dym",
      "variantLabel": "180 g",
      "accentColor": "#5B5247",
      "unitPriceGrosze": 15900,
      "quantity": 2,
      "availableQuantity": 10,
      "lineTotalGrosze": 31800,
      "image": {
        "url": "/products/cedr-i-dym-neutral.webp",
        "alt": "Świeca Cedr i Dym na jasnym tle"
      }
    },
    {
      "variantId": "cmv54jvfz00035e8orvm7uhqo",
      "productSlug": "ambra-nocturna",
      "productName": "Ambra Nocturna",
      "variantLabel": "180 g",
      "accentColor": "#8A5A3C",
      "unitPriceGrosze": 16900,
      "quantity": 1,
      "availableQuantity": 10,
      "lineTotalGrosze": 16900,
      "image": {
        "url": "/products/ambra-nocturna-neutral.webp",
        "alt": "Świeca Ambra Nocturna na jasnym tle"
      }
    }
  ],
  "subtotalGrosze": 48700,
  "removedVariantIds": [
    "nieistniejacy"
  ]
}
```

### Znaczenie kluczy w odpowiedzi

| Klucz | Opis |
| --- | --- |
| `lines[]` | Pozycje, które da się kupić. Ilość jest przycięta do stanu magazynu ( `availableQuantity` , maks. 10). |
| `lineTotalGrosze` | Cena jednostkowa × ilość. |
| `image` | Zdjęcie `NEUTRAL` do miniatury. |
| `subtotalGrosze` | Suma pozycji bez dostawy. |
| `removedVariantIds` | Warianty usunięte z koszyka: nie istnieją, są nieaktywne albo wyprzedane. |

## POST /checkout

`POST` `/checkout`

Tworzy zamówienie `PENDING` z cenami z bazy i PaymentIntent w Stripe. Zwraca `clientSecret` do Stripe Payment Element. Działa dla gościa i zalogowanego klienta (zamówienie jest wtedy przypisane do konta).

### Treść żądania (JSON)

<table>
<tr><th>Parametr</th><th>Wymagany</th><th>Opis</th></tr>
<tr><td><code>locale</code></td><td>tak</td><td><code>pl</code> / <code>en</code>, język maili.</td></tr>
<tr><td><code>email</code></td><td>tak</td><td>E-mail klienta (zapisywany małymi literami).</td></tr>
<tr><td><code>items[]</code></td><td>tak</td><td>1-20 pozycji: <code>variantId</code>, <code>quantity</code> (1-10).</td></tr>
<tr><td><code>shippingMethod</code></td><td>tak</td><td><table><tr><th>Wartość</th><th>Znaczenie</th></tr><tr><td><code>COURIER</code></td><td>kurier, 19,99 zł</td></tr><tr><td><code>PARCEL_LOCKER</code></td><td>paczkomat InPost, 14,99 zł</td></tr></table></td></tr>
<tr><td><code>parcelLockerCode</code></td><td>przy paczkomacie</td><td>Kod paczkomatu, 3-12 liter i cyfr.</td></tr>
<tr><td><code>address</code></td><td>tak</td><td>Adres dostawy, pola jak w adresach.</td></tr>
<tr><td><code>notes</code></td><td>nie</td><td>Uwagi do zamówienia, do 500 znaków.</td></tr>
<tr><td><code>acceptTerms</code></td><td>tak</td><td>Musi być <code>true</code>.</td></tr>
<tr><td><code>website</code></td><td>nie</td><td>Honeypot, musi być pusty.</td></tr>
</table>

### Przykład żądania

```
POST https://lumen.com/api/checkout
Content-Type: application/json

{
  "locale": "pl",
  "email": "anna.nowak@example.com",
  "items": [
    {
      "variantId": "cmv54jvg800085e8opzvup5mw",
      "quantity": 1
    }
  ],
  "shippingMethod": "PARCEL_LOCKER",
  "parcelLockerCode": "WAW21M",
  "address": {
    "firstName": "Anna",
    "lastName": "Nowak",
    "line1": "ul. Przykładowa 12/4",
    "postalCode": "00-950",
    "city": "Warszawa",
    "phone": "+48 600 100 200"
  },
  "notes": "",
  "acceptTerms": true,
  "website": ""
}
```

### Odpowiedź – błąd walidacji (400)

```
{
  "error": "validation_error",
  "issues": [
    {
      "path": "email",
      "code": "invalid_format",
      "message": "Invalid email address"
    },
    {
      "path": "items",
      "code": "too_small",
      "message": "Too small: expected array to have >=1 items"
    },
    {
      "path": "address.postalCode",
      "code": "invalid_format",
      "message": "Invalid string: must match pattern /^\\d{2}-\\d{3}$/"
    },
    {
      "path": "acceptTerms",
      "code": "invalid_value",
      "message": "Invalid input: expected true"
    }
  ]
}
```

### Odpowiedź – brak towaru (409)

```
{
  "error": "cart_changed"
}
```

### Odpowiedź – płatności nieskonfigurowane (503)

```
{
  "error": "payments_unavailable"
}
```

### Odpowiedź – sukces

```
{
  "orderId": "cmv70dzwm000e3e8oco6p858w",
  "orderNumber": "LM-261013-7KQP",
  "clientSecret": "pi_3Q…_secret_…",
  "totalGrosze": 15900
}
```

### Znaczenie kluczy w odpowiedzi

| Klucz | Opis |
| --- | --- |
| `orderId`, `orderNumber` | Utworzone zamówienie. |
| `clientSecret` | Sekret PaymentIntent dla Stripe.js ( `elements({ clientSecret })` ). Nie zapisuj go w logach. |
| `totalGrosze` | Kwota do zapłaty z dostawą. |

Stan magazynu zmniejsza się dopiero przy płatności (webhook). Nieopłacone zamówienie jest anulowane automatycznie po 24 godzinach: API co 15 minut sprawdza stare zamówienia `PENDING` w Stripe, anuluje PaymentIntent i ustawia `CANCELLED`. Jeśli płatność jednak przeszła, zamówienie dostaje `PAID`. Odpowiedź sukcesu pochodzi z kodu i testów, bo w środowisku do zrzutów nie było kluczy Stripe.

# GET /checkout/confirmation

**GET** `/checkout/confirmation`

Strona „dziękujemy" po powrocie ze Stripe. Sprawdza PaymentIntent i zwraca zamówienie. Jeśli płatność już przeszła, a webhook jeszcze nie dotarł, oznacza zamówienie jako opłacone (idempotentnie).

### Query params

| Parametr | Wymagany | Opis |
|---|---|---|
| `paymentIntent` | tak | Id PaymentIntent ( `pi_…` ) z adresu powrotu Stripe. |
| `clientSecret` | tak | Sekret z tego samego adresu. Porównywany w stałym czasie, więc samo `pi_…` nie wystarczy, żeby podejrzeć cudze zamówienie. |

### Przykład żądania

```
GET https://lumen.com/api/checkout/confirmation?paymentIntent=pi_3Q…
&clientSecret=pi_3Q…_secret_…
```

### Odpowiedź – błąd walidacji (400)

```
{
  "error": "validation_error",
  "issues": [
    {
      "path": "paymentIntent",
      "code": "invalid_format",
      "message": "Invalid string: must match pattern /^pi_[A-Za-z0-9]+$/"
    },
    {
      "path": "clientSecret",
      "code": "too_small",
      "message": "Too small: expected string to have >=10 characters"
    }
  ]
}
```

### Odpowiedź – nie znaleziono (404)

```
{
  "error": "order_not_found"
}
```

### Odpowiedź – sukces

Obiekt zamówienia, zob. Obiekt zamówienia.

## POST /webhooks/stripe

**POST** `/webhooks/stripe`

Odbiera zdarzenia Stripe. Treść jest czytana jako surowe bajty, a podpis z nagłówka `Stripe-Signature` weryfikowany sekretem `STRIPE_WEBHOOK_SECRET`.

| Zdarzenie | Efekt |
|---|---|
| `payment_intent.succeeded` | Zamówienie `PAID`, zdjęcie stanów magazynowych, mail z potwierdzeniem. |
| `payment_intent.payment_failed` | Zamówienie `FAILED`. |
| `payment_intent.canceled` | Zamówienie `CANCELLED`. |
| inne | Ignorowane, odpowiedź `{"received": true}`. |

### Przykład żądania

```
stripe listen --forward-to localhost:4300/webhooks/stripe
```

### Odpowiedź – zły podpis (400)

```
{
  "error": "invalid_signature"
}
```

### Odpowiedź – brak kluczy (503)

```
{
  "error": "payments_unavailable"
}
```

### Odpowiedź – sukces

```
{
  "received": true
}
```

Każde zdarzenie jest zapisywane w tabeli `StripeEvent`, więc ponowne doręczenie tego samego zdarzenia niczego nie zmienia.

# POST /auth/register

**POST** `/auth/register`

Zakłada konto klienta i od razu loguje (ustawia `lumen_session`). Wcześniejsze opłacone zamówienia gościa na ten sam e-mail są dopinane do konta.

### Treść żądania (JSON)

| Parametr | Wymagany | Opis |
|---|---|---|
| `name` | tak | Imię i nazwisko, 1-80 znaków. |
| `email` | tak | E-mail, zapisywany małymi literami. |
| `password` | tak | 10-128 znaków. Hashowane argon2id. |
| `locale` | nie | `pl` / `en`. |
| `acceptTerms` | tak | `true`. |
| `website` | nie | Honeypot. |

### Przykład żądania

```
POST https://lumen.com/api/auth/register
Content-Type: application/json

{
  "name": "Anna Nowak",
  "email": "anna.nowak@example.com",
  "password": "…",
  "locale": "pl",
  "acceptTerms": true
}
```

### Odpowiedź – błąd walidacji (400)

```
{
  "error": "validation_error",
  "issues": [
    {
      "path": "name",
      "code": "too_small",
      "message": "Too small: expected string to have >=1 characters"
    },
    {
      "path": "email",
      "code": "invalid_format",
      "message": "Invalid email address"
    },
    {
      "path": "password",
      "code": "too_small",
      "message": "Too small: expected string to have >=10 characters"
    },
    {
      "path": "acceptTerms",
      "code": "invalid_value",
      "message": "Invalid input: expected true"
    }
  ]
}
```

### Odpowiedź – e-mail zajęty (409)

```
{
  "error": "email_taken"
}
```

### Odpowiedź – sukces (201)

```
{
  "user": {
    "id": "cmv70dzu600003e8oyxrffhyc",
    "email": "anna.nowak@example.com",
    "name": "Anna Nowak",
    "role": "CUSTOMER"
  }
}
```

### Znaczenie kluczy w odpowiedzi

| Klucz | Opis |
| --- | --- |
| `user` | Publiczne dane użytkownika: `id`, `email`, `name`, `role` (`CUSTOMER` / `ADMIN`). Rola nigdy nie pochodzi z żądania. |

## POST /auth/login · logout · GET /auth/me

POST `/auth/login`

POST `/auth/logout`

GET `/auth/me`

Logowanie, wylogowanie i odczyt bieżącej sesji. Przy nieistniejącym e-mailu serwer i tak liczy hash, żeby czas odpowiedzi nie zdradzał, czy konto istnieje.

### Treść żądania /auth/login (JSON)

| Parametr | Wymagany | Opis |
|---|---|---|
| `email` | tak | E-mail. |
| `password` | tak | Hasło, do 128 znaków. |

### Przykład żądania

```
POST https://lumen.com/api/auth/login
Content-Type: application/json

{"email": "anna.nowak@example.com", "password": "…"}
```

### Odpowiedź – złe dane (401)

```
{
  "error": "invalid_credentials"
}
```

### Odpowiedź – sukces

```
Set-Cookie: lumen_session=…; Path=/; Expires=Thu, 12 Nov 2026 18:23:35 GMT;
HttpOnly; SameSite=Lax

{
  "user": {
    "id": "cmv70dzu600003e8oyxrffhyc",
    "email": "anna.nowak@example.com",
    "name": "Anna Nowak",
    "role": "CUSTOMER"
  }
}
```

### GET /auth/me – zalogowany

```
{
  "user": {
    "id": "cmv70dzu600003e8oyxrffhyc",
    "email": "anna.nowak@example.com",
    "name": "Anna Nowak",
    "role": "CUSTOMER"
  }
}
```

### GET /auth/me – bez sesji

```
{
  "user": null
}
```

### POST /auth/logout – sukces

```
Set-Cookie: lumen_session=; Max-Age=…; Path=/; Expires=Thu, 01 Jan 1970 00:00:00
GMT; SameSite=Lax

{
  "ok": true
}
```

`/auth/me` nigdy nie zwraca 401, tylko `{"user": null}`, żeby front mógł bez błędów sprawdzić stan logowania.

# POST /auth/password-reset/request · confirm

POST `/auth/password-reset/request`

POST `/auth/password-reset/confirm`

Reset hasła mailem. Link jest ważny 1 godzinę i działa jeden raz.

### Treść żądania /request (JSON)

| Parametr | Wymagany | Opis |
|---|---|---|
| `email` | tak | E-mail konta. |
| `locale` | nie | Język maila i adresu strony z formularzem. |
| `website` | nie | Honeypot. |

### Treść żądania /confirm (JSON)

| Parametr | Wymagany | Opis |
|---|---|---|
| `token` | tak | Token z linku w mailu (20-200 znaków). |
| `password` | tak | Nowe hasło, 10-128 znaków. |

### Przykład żądania

```
POST https://lumen.com/api/auth/password-reset/request
Content-Type: application/json

{"email": "anna.nowak@example.com", "locale": "pl"}
```

### /request – odpowiedź (202, zawsze ta sama)

```
{
  "ok": true
}
```

### /confirm – zły lub zużyty token (400)

```
{
  "error": "invalid_token"
}
```

### /confirm – sukces

```
{
  "ok": true
}
```

`/request` odpowiada tak samo dla istniejących i nieistniejących kont. Po udanym `/confirm` wszystkie sesje użytkownika są usuwane.

# GET /account/orders · /account/orders/:id

GET `/account/orders` *wymaga sesji*

GET `/account/orders/:id` *wymaga sesji*

Historia zamówień zalogowanego klienta (bez `PENDING` i `FAILED`, maks. 50 najnowszych) i szczegóły jednego zamówienia.

### Parametry ścieżki

| Parametr | Wymagany | Opis |
|---|---|---|
| `id` | tak (szczegóły) | Id zamówienia. Cudze zamówienie daje 404, nie 403. |

### Przykład żądania

```
curl -b cookies.txt https://lumen.com/api/account/orders
```

### Odpowiedź – bez sesji (401)

```
{
  "error": "unauthorized"
}
```

### Odpowiedź – nie znaleziono (404)

```
{
  "error": "order_not_found"
}
```

### Odpowiedź – sukces /account/orders (skrócony przykład)


```
[
  {
    "id": "cmv70dzwm000e3e8oco6p858w",
    "number": "LM-261011-G5QE",
    "status": "PAID",
    "email": "anna.nowak@example.com",
    "createdAt": "2026-10-11T14:22:58.820Z",
    "paidAt": "2026-10-11T14:23:58.820Z",
    "subtotalGrosze": 72700,
    "shippingGrosze": 0,
    "totalGrosze": 72700,
    "trackingNumber": null,
    "shipping": {
      "method": "COURIER",
      "parcelLockerCode": null,
      "firstName": "Anna",
      "lastName": "Nowak",
      "line1": "ul. Przykładowa 12/4",
      "line2": "",
      "postalCode": "00-950",
      "city": "Warszawa",
      "country": "PL",
      "phone": "+48 600 100 200"
    },
    "items": [
      {
        "productName": "Tytoń i Wanilia",
        "variantLabel": "180 g",
        "unitPriceGrosze": 16900,
        "quantity": 1
      },
      {
        "productName": "Ambra Nocturna",
        "variantLabel": "380 g",
        "unitPriceGrosze": 27900,
        "quantity": 2
      }
    ]
  },
  "…"
]
```


Pola opisuje Obiekt zamówienia.

# /account/addresses

GET `/account/addresses` *wymaga sesji*

POST `/account/addresses` *wymaga sesji*

PUT `/account/addresses/:id` *wymaga sesji*

DELETE `/account/addresses/:id` *wymaga sesji*

Zapisane adresy klienta (maks. 10). Pierwszy adres staje się domyślnym. Ustawienie `isDefault: true` zdejmuje tę flagę z pozostałych.

### Treść żądania POST / PUT (JSON)

| Parametr | Wymagany | Opis |
|---|---|---|
| `firstName` | tak | Imię, 1-60 znaków. |
| `lastName` | tak | Nazwisko, 1-80 znaków. |
| `line1` | tak | Ulica i numer, 1-120 znaków. |
| `line2` | nie | Dodatkowa linia adresu, do 120 znaków. |
| `postalCode` | tak | Kod pocztowy w formacie `00-000`. |
| `city` | tak | Miejscowość, 1-80 znaków. |
| `country` | nie | Zawsze `PL` (wysyłka tylko w Polsce). |
| `phone` | tak | Telefon, 9-20 znaków: cyfry, spacje, nawiasy, myślniki, opcjonalny `+`. |
| `isDefault` | nie | Czy adres ma być domyślny. |

### Przykład żądania

```
POST https://lumen.com/api/account/addresses
Content-Type: application/json

{
  "firstName": "Anna",
  "lastName": "Nowak",
  "line1": "ul. Ogrodowa 5",
  "postalCode": "30-001",
  "city": "Kraków",
  "phone": "+48 600 100 200",
  "isDefault": false
}
```

### Odpowiedź – błąd walidacji (400)

```
{
  "error": "validation_error",
  "issues": [
    {
      "path": "postalCode",
      "code": "invalid_format",
      "message": "Invalid string: must match pattern /^\\d{2}-\\d{3}$/"
    },
    {
      "path": "phone",
      "code": "invalid_format",
      "message": "Invalid string: must match pattern /^\\+?[0-9 ()-]{9,20}$/"
    }
  ]
}
```

### Odpowiedź – limit adresów (409)

```
{
  "error": "address_limit"
}
```

### POST – sukces (201)


```
{
  "id": "cmv70esd3000wp98oox3p2z1e",
  "firstName": "Anna",
  "lastName": "Nowak",
  "line1": "ul. Ogrodowa 5",
  "line2": "",
  "postalCode": "30-001",
  "city": "Kraków",
  "country": "PL",
  "phone": "+48 600 100 200",
  "isDefault": false
}
```


### GET – sukces


```
[
  {
    "id": "cmv70dzv200013e8o6nhp90k0",
    "firstName": "Anna",
    "lastName": "Nowak",
    "line1": "ul. Przykładowa 12/4",
    "line2": "",
    "postalCode": "00-950",
    "city": "Warszawa",
    "country": "PL",
    "phone": "+48 600 100 200",
    "isDefault": true
  }
]
```


### DELETE – sukces

```
HTTP/1.1 204 No Content
```

Każda operacja sprawdza, czy adres należy do zalogowanego klienta. Cudzy adres daje `404 address_not_found`.

# GET /admin/stats

**GET** `/admin/stats` *wymaga sesji admina*

Liczniki na pulpit panelu.

### Przykład żądania

```
curl -b admin-cookies.txt https://lumen.com/api/admin/stats
```

### Odpowiedź – bez roli admina (403)

```
{
  "error": "forbidden"
}
```

### Odpowiedź – sukces

```
{
  "ordersToday": 0,
  "paidRevenueGrosze30d": 298700,
  "pendingShipment": 3,
  "lowStockVariants": 2
}
```

### Znaczenie kluczy w odpowiedzi

| Klucz | Opis |
|---|---|
| `ordersToday` | Zamówienia z dzisiaj (bez `PENDING` i `FAILED`). |
| `paidRevenueGrosze30d` | Przychód z opłaconych zamówień z 30 dni. |
| `pendingShipment` | Zamówienia `PAID` czekające na wysyłkę. |
| `lowStockVariants` | Aktywne warianty z co najwyżej 5 sztukami. |

## GET /admin/products · PATCH products · variants

**GET** `/admin/products` *wymaga sesji admina*

**PATCH** `/admin/products/:id` *wymaga sesji admina*

**PATCH** `/admin/variants/:id` *wymaga sesji admina*

Lista wszystkich produktów (także `DRAFT` i `ARCHIVED`) z wariantami i stanami. Edycja statusu, wyróżnienia, koloru, ceny i stanu magazynu.

### Treść PATCH /admin/products/:id (JSON, wszystkie pola opcjonalne)

| Parametr | Wymagany | Opis |
|---|---|---|
| `status` | nie | <table><tr><th>Wartość</th><th>Znaczenie</th></tr><tr><td>`ACTIVE`</td><td>widoczny w sklepie</td></tr><tr><td>`DRAFT`</td><td>szkic, ukryty</td></tr><tr><td>`ARCHIVED`</td><td>wycofany, ukryty</td></tr></table> |
| `featured` | nie | Wyróżnienie na stronie głównej. |
| `accentColor` | nie | Kolor w formacie `#RRGGBB`. |

### Treść PATCH /admin/variants/:id (JSON, wszystkie pola opcjonalne)

| Parametr | Wymagany | Opis |
|---|---|---|
| `priceGrosze` | nie | Cena w groszach, 100-1 000 000. |
| `stock` | nie | Stan magazynu, 0-100 000. |
| `active` | nie | Czy wariant jest w sprzedaży. |

### Przykład żądania

```
PATCH https://lumen.com/api/admin/variants/cmv54jvfz00035e8orvm7uhqo
Content-Type: application/json

{"stock": 24, "priceGrosze": 16900}
```

### Odpowiedź – błąd walidacji (400)

```
{
  "error": "validation_error",
  "issues": [
    {
      "path": "accentColor",
      "code": "invalid_format",
      "message": "Invalid string: must match pattern /^#[0-9a-fA-F]{6}$/"
    }
  ]
}
```

### Odpowiedź – nie znaleziono (404)

```
{
  "error": "variant_not_found"
}
```

### PATCH – sukces

```
{
  "ok": true
}
```

### GET /admin/products – sukces (skrócony przykład)


```
[
  {
    "id": "cmv54jvex00005e8obintkd10",
    "slug": "ambra-nocturna",
    "name": "Ambra Nocturna",
    "family": "WOODY",
    "accentColor": "#8A5A3C",
    "status": "ACTIVE",
    "featured": true,
    "variants": [
      {
        "id": "cmv54jvfz00035e8orvm7uhqo",
        "sku": "LUM-AMBRA-NOCTURNA-180",
        "weightGrams": 180,
        "burnHours": 45,
        "priceGrosze": 16900,
        "stock": 24,
        "active": true
      },
      {
        "id": "cmv54jvg100045e8obme4e1xi",
        "sku": "LUM-AMBRA-NOCTURNA-380",
        "weightGrams": 380,
        "burnHours": 85,
        "priceGrosze": 27900,
        "stock": 12,
        "active": true
      }
    ]
  },
  "…"
]
```

## GET /admin/orders · /admin/orders/:id

**GET** `/admin/orders` *wymaga sesji admina*

**GET** `/admin/orders/:id` *wymaga sesji admina*

Lista zamówień ze stronicowaniem (20 na stronę), filtrem statusu i wyszukiwarką. Bez filtra statusu lista pomija `PENDING`.

### Query params

| Parametr | Wymagany | Opis |
|---|---|---|
| `status` | nie | Jeden ze statusów:<br>`PENDING` — utworzone, czeka na płatność (nie widać go na koncie klienta ani na domyślnej liście w panelu)<br>`PAID` — opłacone, do wysłania<br>`SHIPPED` — wysłane, ma numer przesyłki<br>`DELIVERED` — doręczone<br>`CANCELLED` — anulowane: w panelu, automatycznie po 24 h bez płatności albo po anulowaniu PaymentIntent w Stripe<br>`FAILED` — płatność odrzucona |
| `query` | nie | Szukanie w numerze zamówienia, e-mailu i nazwisku (bez rozróżniania wielkości liter), do 120 znaków. |
| `page` | nie | Numer strony, od 1 (domyślnie 1). |

### Przykład żądania

```
GET https://lumen.com/api/admin/orders?status=PAID&query=nowak&page=1
```

### Odpowiedź – sukces (skrócony przykład)


```
{
  "items": [
    {
      "id": "cmv70dzwq000h3e8oeji6npgx",
      "number": "LM-261012-XXT3",
      "email": "kasia.lewandowska@example.com",
      "status": "CANCELLED",
      "totalGrosze": 42800,
      "itemCount": 2,
      "createdAt": "2026-10-12T16:22:58.825Z"
    },
    {
      "id": "cmv70dzwm000e3e8oco6p858w",
      "number": "LM-261011-G5QE",
      "email": "anna.nowak@example.com",
      "status": "PAID",
      "totalGrosze": 72700,
      "itemCount": 3,
      "createdAt": "2026-10-11T14:22:58.820Z"
    },
    "…"
  ],
  "total": 6,
  "page": 1,
  "pageSize": 20
}
```


### Znaczenie kluczy w odpowiedzi

| Klucz | Opis |
|---|---|
| `items[]` | Zamówienia: `id`, `number`, `email`, `status`, `totalGrosze`, `itemCount` (suma sztuk), `createdAt`. |
| `total` | Liczba wszystkich pasujących zamówień. |
| `page`, `pageSize` | Bieżąca strona i jej rozmiar. |

### GET /admin/orders/:id

Obiekt zamówienia (zob. Obiekt zamówienia) z dodatkowym polem `notes` (uwagi klienta).

## PATCH /admin/orders/:id/status

**PATCH** `/admin/orders/:id/status` *wymaga sesji admina*

Zmiana statusu zamówienia. Dozwolone przejścia są sprawdzane po stronie serwera.

### Treść żądania (JSON)

| Parametr | Wymagany | Opis |
|---|---|---|
| `status` | tak | Nowy status. |
| `trackingNumber` | przy `SHIPPED` | Numer przesyłki, do 64 znaków. Wymagany przy wysyłce, jeśli zamówienie go jeszcze nie ma. |

| Z | Dozwolone na |
|---|---|
| `PENDING` | `CANCELLED` |
| `PAID` | `SHIPPED`, `CANCELLED` (anulowanie opłaconego przywraca stany magazynowe) |
| `SHIPPED` | `DELIVERED` |
| `DELIVERED`, `CANCELLED`, `FAILED` | brak (status końcowy) |

### Przykład żądania

```
PATCH https://lumen.com/api/admin/orders/cmv70dzwm000e3e8oco6p858w/status
Content-Type: application/json

{"status": "SHIPPED", "trackingNumber": "62000000000000000001"}
```

### Odpowiedź – brak numeru przesyłki (400)

```
{
  "error": "tracking_required"
}
```

### Odpowiedź – niedozwolone przejście (400)

```
{
  "error": "invalid_transition"
}
```

### Odpowiedź – sukces (skrócony przykład)

```
{
  "id": "cmv70dzwm000e3e8oco6p858w",
  "number": "LM-261011-G5QE",
  "status": "SHIPPED",
  "trackingNumber": "6200000000000000000001",
  "totalGrosze": 72700,
  …
}
```

Pełna odpowiedź to obiekt zamówienia po zmianie.